

CTF专辑--花指令

一、垃圾代码 (JunkCode)

二、汇编指令的等价替换

(1) 算数拆分

(2) 等价解释

I .push/pop

II .jmp addr

III.call addr

IV.retn

V.enter/leave

VI.xor A,A

VII.add/sub A,1

VIII.push/popad

IX.and

X.call 0h

XI.mov

(3) 间接赋值

三、抵消型花指令

(1) 运算抵消

I .运算抵消实例

(2) 栈平衡实例

I .push/pop类

II .动态使栈平衡

四、jmp db型花指令

(1) 无条件跳转的jmp db型

(2) 条件跳转的jmp db型

(3) jmp to db干扰分析型(jx+jnx)

五、call 0h型

(1) call +db+平衡

(2) call + add [esp], n + retn

六、总结

一、垃圾代码（JunkCode）

对于垃圾代码和花指令的区分，我们一般将没有实际意义的代码称作垃圾代码（Junk Code/Garbage code），而把干扰调试器分析的代码称作花指令。

典型的JunkCode一般是一些操作指令具有相同的操作数，执行一些毫无作用的运算，比如：

```
1  mov  eax,eax;
2  xchg  esp,esp;
3  jmp  rva=0;//E9 00 00 00 00
4  xor  eax,0;      ; 任何一个数异或0等于本身
```

二、汇编指令的等价替换

（1）算数拆分

所谓的算数拆分，就是将原来的操作数，拆分为多次进行计算，比如：

```

1  add eax,0x5;
2  就可以拆分为
3  add eax,0x2;
4  add eax,0x1;
5  inc eax;
6  inc eax;
7  再如:
8  shl eax,5
9  就可以拆分为:
10 shl eax,3;
11 shl eax,1;
12 shl eax,1;
13 当然,也可以综合其他寄存器进行值的传递,比如:
14 add eax,4;
15 可以改写成:
16 xor ebx,ebx;
17 add ebx,2;
18 mul ebx,2;
19 add eax,ebx;

```

诸如此类的可以写很多种,不再阐述

下面对几个指令的等价解释进行描述:

(2) 等价解释

1 .push/pop

对于push

```

1  push 0x11111;
2  在x86程序里就等价于
3  sub esp,0x4;
4  mov [esp],0x11111;

```

而对于pop

```

1  pop eax;
2  在x86程序里面就等价于
3  mov eax,[esp];
4  add esp,0x4;
5  //因此，当某个寄存器没有实际作用的时候，可以用多个Pop,来代替add esp;

```

II .jmp addr

对于跳转，不做过多解释，本质上是

```

1  mov eip,addr;

```

当然，它常用作这种变型在花指令里面使用：

```

1  push addr;
2  retn;

```

III.call addr

对于call指令，需要各位重视的是，它等价于

```

1  push next opcodeAddr;
2  jmp addr;

```

这边注意的是，它push了下一个指令的地址，也就是函数返回后的地址，这个地址的作用，可以实现控制流劫持，比如经常在pwn里面出现的栈溢出漏洞，基本原理就是通过修改栈上的这个地址，来getshell。

IV.retn

retn一般出现在call进的函数里面，一般用作函数的返回，当然，其后面也可以接值。

```

1  比如retn 0x5
2  其等价于:
3  mov eip,[esp];
4  add esp,0x4+0x5;
```

下面给出一种常见的等价变换:

```

1  push ecx;
2  mov ecx,[esp+4];
3  add esp,8;
4  jmp ecx;
5  //这种写法会破坏ecx的值, 因此使用的前提是ecx没有正在被程序使用
```

当然除了上面基本的几个等价指令的解释, 下面的几个指令也是汇编中很常见的:

V.enter/leave

对于enter指令的定义, 官方的定义是这样的:

ENTER numbytes, nestinglevel

ENTER 有两个操作数:

第一个是常数, 定义为局部变量保存的堆栈空间字节数; 第二个定义了过程的词法嵌套级。

它为局部变量保留堆栈空间, 把 EBP 入栈。具体来说, 它执行三个操作:

- 把 EBP 入栈 (push ebp)
- 把 EBP 设置为堆栈帧的基址 (mov ebp, esp)
- 为局部变量保留空间 (sub esp, numbytes)

下面给出一个实例, ENTER 指令为局部变量保留了 8 个字节的堆栈空间:

```
enter 8,0
```

它与下列指令等效:

```

1  push ebp;
2  mov ebp,esp;
3  sub esp,8;
```

而一般情况下，只是简单的enter 后面不接参数，那就没有sub esp的说法了

而leave指令则跟enter相反，一般出现在retn之前：

它等价于：

```
1  mov esp, ebp;  
2  pop ebp;
```

VI.xor A,A

```
1  等价于 mov A,0;  
2  因为异或同一个值的结果必定是0;
```

VII.add/sub A,1

```
1  等价于  
2  inc A;//A+=1  
3  dec A;//A-=1
```

VIII.push/popad

PUSHAD 指令按照 EAX、ECX、EDX、EBX、ESP（执行 PUSHAD 之前的值）、EBP、ESI 和 EDI 的顺序，将所有 32 位通用寄存器压入堆栈。

IX.and

常用来实现栈对齐，比如,实现16位对齐：

```
and esp,0xffffffff
```

X.call 0h

16进制为：E8 00 00 00 00

这种指令常用来获取下一行代码的地址，也可用于获取eip的值。

▼

⌂ | 复制代码

```
1 004710D5 > E8 00000000 call 11111.004710DA
2 004710DA |. 68 10B14900 push 11111.0049B110
3 ▼ 这里执行call后, eip的值为004710DAh, 而此时[esp]的值, 就是0x4710DAh, 因此获取call时的eip就是[esp]-0x5;
```

Xl.mov

▼

⌂ | 复制代码

```
1 mov op1,op2 ----> push op2 / pop op1
```

(3) 间接赋值

我们知道，数学上常有 $b=a, c=b$ 就是 $c=a$

那么同样的，在构建花指令的时候也可以这么做。

三、抵消型花指令

抵消型花指令主要的注意点有两个方面，一个是运算的抵消，即逆运算；

另外一个就是栈平衡类型，栈平衡也是考虑堆栈抵消型花指令的注意点。

抵消型花指令一般不会干扰分析引擎分析，但可能会影响栈平衡

(1) 运算抵消

常见的运算抵消有这样几种

push-pop (压栈-出栈)

add-sub adc-sbb (加法-减法 进位加法-借位减法)

mul-div imul-idiv (乘法-除法 整数乘法-整数除法)

inc-dec (加1-减1)

shl-shr (左移-右移)

xor-xor (异或)

not-not (取反)

I. 运算抵消实例

```
1  push eax;  
2  pop  eax;  
3  add  eax,2;  
4  mul  eax,2;  
5  div  eax,2;  
6  xor  eax,2;  
7  xor  eax,2;  
8  inc  eax;  
9  dec  eax;  
10 inc  ebx;  
11 inc  ebx;  
12 sub  ebx,2;  
13 shl  ebx,2;  
14 shr  ebx,2;  
15 sub  eax,2;
```

(2) 栈平衡实例

I. .push/pop类

```
1  pushad;  
2  pushfd;  
3  push  eax;  
4  push  ecx;  
5  push  ebx;  
6  push  esi;  
7  pop   esi;  
8  pop   ebx;  
9  pop   ecx;  
10 pop   eax;  
11 popfd;  
12 popad;
```


可以观察到的是，这些push、pop指令都遵循基本的出入栈规则，即后进先出；因此编写这些花指令的时候尤其需要注意。

II.动态使栈平衡

动态使栈平衡的主要思路就是保存原来的栈指针,后面再恢复；

下面给出一个实例：

```
1  push ebp;
2  mov  ebp,esp;
3  sub  esp,1000h;
4  push 0h;
5  push esi;
6  push eax;
7  mov  esp,ebp;
8  pop  ebp;
```

复制代码

四、jmp db型花指令

这类花指令主要是干扰分析工具分析：

其中IDA 使用的反汇编算法是Recursive Traversal

Olldbg 使用的反汇编算法是LinearSweep/Recursive Traversal(按ctrl+A组合键时)

其中线性扫描（LinearSweep）算法的技术含量不高，反汇编工具将整个模块中的每一条指令都反汇编成汇编指令，将遇到的机器码都作为代码处理，没有对所反汇编的内容进行任何判断。因此，该种算法不能有效的将代码和数据区分开，从而导致反汇编出现错误。这种错误会影响对下一条指令的正确识别，会使整个反汇编都出现错误。

而递归行进法（Recursive Traversal）按照代码可能执行的顺序来反汇编程序，对每条可能的路径进行扫描。当解码出分支指令后，反汇编工具就将这个地址记录下来，并分别反汇编各个分支中的指令，因此该算法灵活度高。

(1) 无条件跳转的jmp db型

最基本的Jmp db型就是跳转接上了一个干扰的db opcode：

例如

```
1 004710D5 > $ /EB 03      jmp short 11111.004710DA
2 004710D7      |55      db 55                ; CHA
  R 'U'
3 004710D8      |E8      db E8                ; 注
  意这里
4 004710D9      . |AA      db AA
5 004710DA      > \83C0 02  add eax,0x2
```

就会被分析成：

```
1 004710D5 > /EB 03      jmp short 11111.004710DA
2 004710D7      |55      push ebp                ; kern
  e132.768E00C9
3 004710D8      |E8 AA83C002  call 03079487
```

说明：Linear Sweep型反汇编引擎是逐行反汇编的，该花指令中的垃圾数据db EBh 干扰了其工作，因此错误地确定了指令的起始位置，导致反汇编的一些跳转指令的地址无效

而OD使用递归引擎分析就会正常，变成：

```
1 004710D5 > $ /EB 03      jmp short 11111.004710DA
2 004710D7      |55      db 55                ; CHA
  R 'U'
3 004710D8      |E8      db E8
4 004710D9      . |AA      stos byte ptr es:[edi]
5 004710DA      > \83C0 02  add eax,0x2
```

(2) 条件跳转的jmp db型

条件跳转跟无条件跳转其实差不多，主要的是用条件跳转代替了原来的jmp，

下面给出一个CTF中常出现的实例：

```

1  start:
2      xor eax,eax;
3      test eax,eax;
4      jz label1;
5      jnz label1;
6      db E8h;
7  label 1:
8      xor eax,3;
9      add eax,3;
10     .....

```

这里构建的前面四个指令xor\test\jz\jnz可以构成一个控制流的改变，因为上面xor和test的原因，导致下面的跳转必定只有一个成立，所以，在构建汇编代码的时候也可以用这种方法来增加分析难度，增加无效分支结构和跳转。

(3) jmp to db干扰分析型(jx+jnx)

注：这里的jmp to db指的是一类，控制转移指令到的地址是无效的数据(db)的情况

这里的jx,jnx一般指两个相对的条件跳转，上面常接

- (1) stx/clx;
- (2) xor ebx,ebx;test ebx,ebx;
- (3) xor reg,value1;cmp reg,value1;

来设置位，让条件跳转成立。

在递归算法中一个十分重要的假设是：对任意一条控制转移指令（jmp/retn/call等）都能确定其后继（即转移）的目的地址，要想要迷惑这类反汇编工具，只要让其难以确定跳转的目的地址即可；

一个实例如下：

```

1  start:
2      xor eax,eax;
3      test eax,eax;
4      jz label0;
5      jnz label1;
6  label 0:
7      db 0E8h
8  label 1:
9      xor eax,3;
10     add eax,3;
11     .....

```

与上面的实例2作对比，可以看见是两个跳转实际上是跳到了不同的地方，一个是混淆的代码，一个是真正的控制流，而上面只是直接跳到了真正的控制流，只不过是控制流上面接了脏字节。

OD中反编译的结果如下

```

1  004710D5 > 33C0      xor eax,eax
2  004710D7      85C0      test eax,eax
3  004710D9      74 03     je short 11111.004710DE
4  004710DB      75 00     jnz short 11111.004710DD
5  004710DD      E8 83F00383 call 834B0165
6  004710E2      c003 90   rol byte ptr ds:[ebx],0x90
7

```

这里具体解释下原因，首先是有两个跳转

je跳转到的是xor eax,3而jnz是跳转到db E8，而E8是call的opcode 后面需要4个字节作为地址，所以这边就会将xor等的opcode 作为地址，从而导致识别错误。

因此这种写法，在CTF中非常常见，因为两种引擎都不能将他分析出来，需要手动去除。

注：（这里的db，也可以换成一些干扰堆栈平衡的指令，不过一定不会被执行，比如add esp,0x1）

五、call 0h型

上面我们提到了call 00h可以获取eip，但因为这里的call的rva是0，所以说，它的出现不会影响正常的控制流，因此，我们可以出现一些无用的call 0h和混淆的字节。

(1) call +db+平衡

```
1  call loc+1;
2  db C8;
3  pop eax;//add esp,0x4;
```

这里为了确保堆栈还原，pop当然也可以换成add esp,0x4;中间的脏字节也可以任意加（call的偏移也要相应的更改）

(2) call + add [esp], n + retn

下面给出一个易语言自带的花指令的实例：

```
1  004010BF  .  E8 00000000  call 1111.004010C4
2  004010C4  /$ 830424 06  add dword ptr ss:[esp],0x6
3  004010C8  \.  C3  retn
4  004010C9  B9  db B9
```

可以看到这里的add，是将call 00h的返回的地址，也就是[esp]+0x6（add 的四个字节+C3 B9）来实现一个跳转，到004010CA，对于这种，只需要将下面的特征码patch掉就可以了：

E80000000083042406C3??

六、总结

花指令的使用远不止如此，CTF中的花指令一般就上述的情况，而像APT，复杂的恶意文件涉及到的花指令，就不止这些了，还会涉及到不同标志寄存器的改变，不同代码块之间的逻辑，对于这些，则要靠逆向工程师的基本功水平，和对不同调试工具的熟悉。